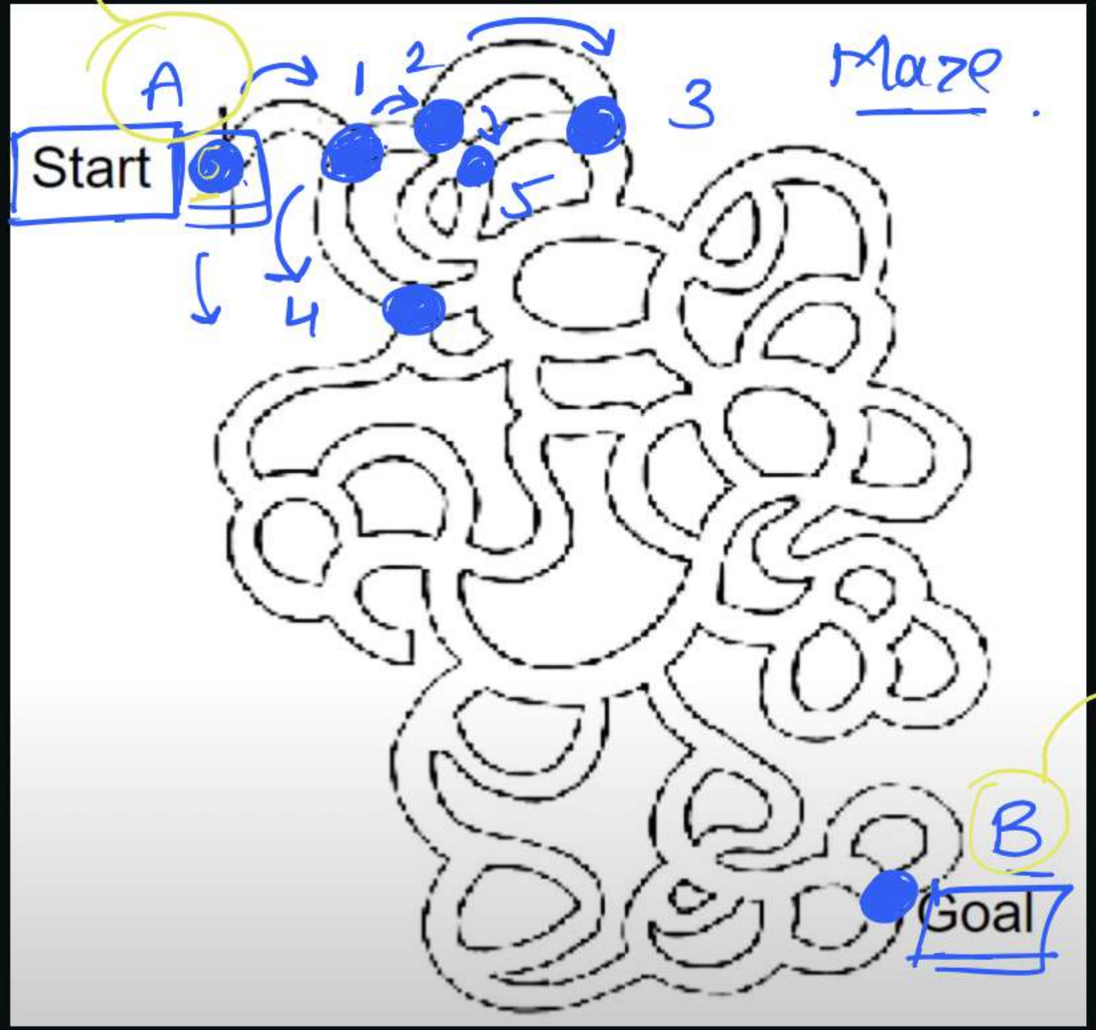
Search :-

↳ Exploring a space of possible solutions to find the most suitable or optimal one for a given problem.

↳ The agents that help us to solve a problem are called as problem-solving agents.

eg. a maze, in which we start ~~..~~ with point A & we need to reach point B.

Starting / Initial State.



Goal / Final State(S)

Problem solving process :-

1. Goal formulation :

↳ The agent adopts the goal to reach B. in the maze.

2. <u>Problem formulation</u> :-

↳ Finalizing the $\boxed{\text{States}}$ & $\boxed{\text{actions}}$ necessary to reach the goal.

↳ In the above eg, action can be, moving from a node to its adjecent ones.

3. <u>Search</u> :-

↳ This will be the process prior to the action.

↳ Simulate the sequence of actions in the model & find the one that reaches the $\boxed{\text{goal.}}$ "there can be multiple goals".

↳ That path is called a "<u>Solution</u>".

4. <u>Execution</u> :-

↳ Executing the actions in the Solution, one by one.

Deterministic & Non-deterministic Env. :-

1. Deterministic Env. :-

- When an outcome of an action is predefined & fully predictable. → Predictability
- Next state can be determined with certainty. → Certainty
- Performing the same action in the same state will always lead to the same result. → Reproducibility.

eg. playing chess.

- "given the state of board & a player's move → result will always be the same".

2. Non-deterministic Env. :-

↳ Uncertain

↳ multiple possible outcomes.

↳ Probabilistic nature.

eg. Driving a car in traffic ; weather predictions.

↳ obstacles, traffic signals.

$\boxed{A , B , C}$

Open-loop v/s Closed-loop :-

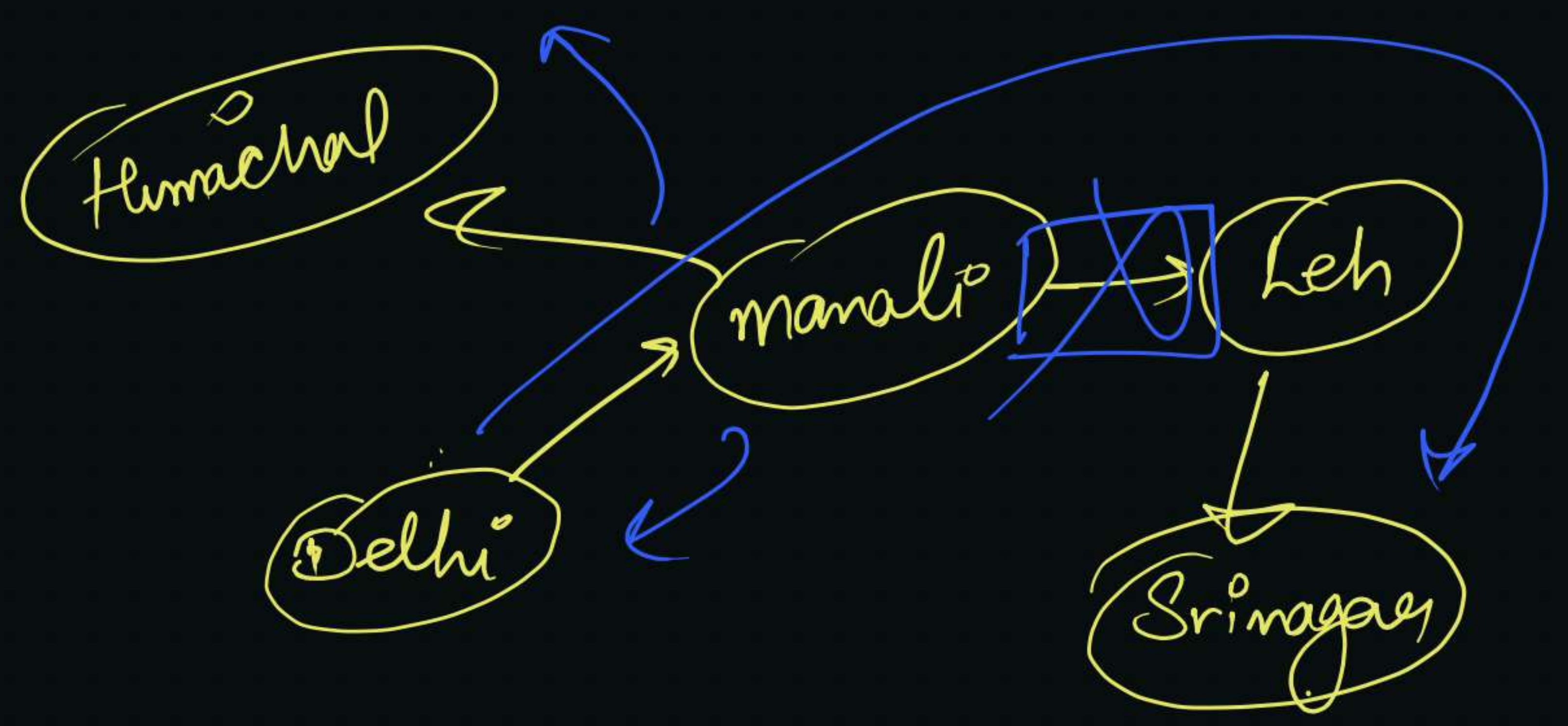
1. Open-loop :- → (D)

↳ In deterministic, known environments., the solution of any problem is the fixed sequence of action.

eg. A → 1, 1 → 2, 2 → 4, ...

↳ In a correct model, "if the agent has the solution", then it can ignore its percepts while executing the actions. → Because the sol^n is guaranteed.

} execution step.



2. Closed-loops :-

↳ This is associated with non-deterministic envr.

↳ Solution will be a branching strategy (dynamic), where actions depends upon the percepts arrived.

Search Problem :- Formal definition of a search problem.

1. State Space :-

↳ set of the sequence of all possible states.

2. Actions :-

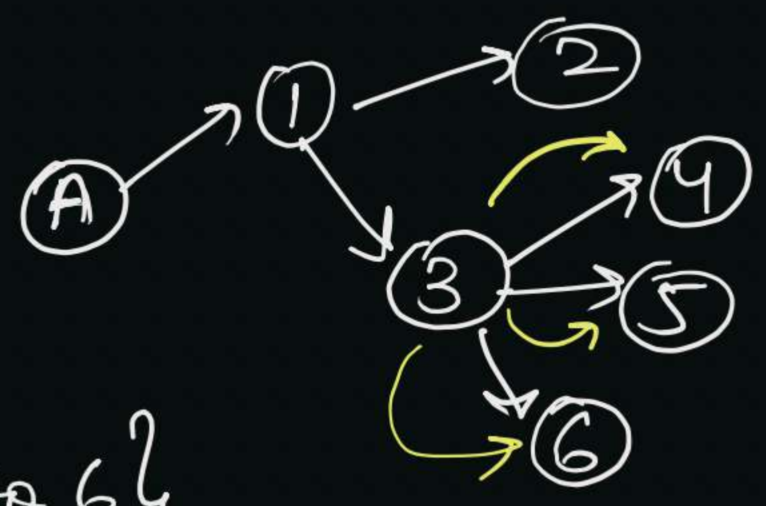
↳ Given a state s, Actions(S) ⟶ finite set of actions that can be performed from S.

Actions(A) = {to 1}

Actions(1) = {to 2, to 3}.

Action(2) = { }

Actions(3) = { to 4 , to 5, to 6}.



3. Transition Model :-

↳ It describes what each action does,

Result(s, a) ⟶ returns the state that results from doing "a" on state "s".

eg. $\boxed{\text{Result}(A, \text{to } 1) = \boxed{1}}$

4. Action Cost function :-

↳ Numeric cost of applying action "a" in state "s" to reach s'.

$\boxed{c(s, a, s') = ?}$

↳ Used as a performance measure.

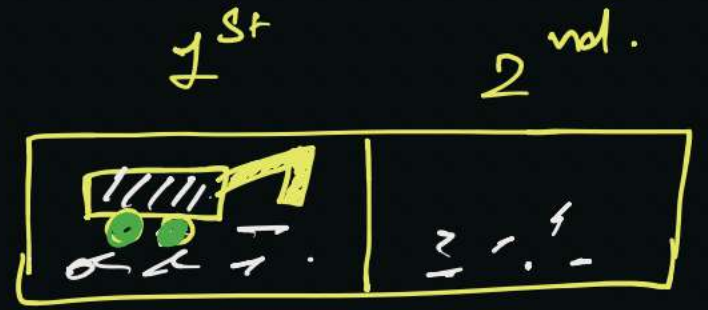
eg. Shortest route-finding agents ; cost → "distance b/w the states."

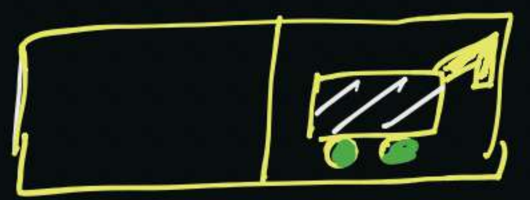
Conclusion :-

- Sequence of actions form a $\boxed{\text{path}}$.
- $\boxed{\text{Solution}}$ is a path from initial → final state.
- $\boxed{\text{Total cost}}$, generally, sum of individual action costs,
- $\boxed{\text{"Optimal Solution"}}$ → Solution with lowest total cost.

HW

eg. Consider a two-cell vacuum world problem,

Initial State → 

; Final State → 

Cells contain trash, agent can move left, righ or Suck the trash. Create a State - Space graph.